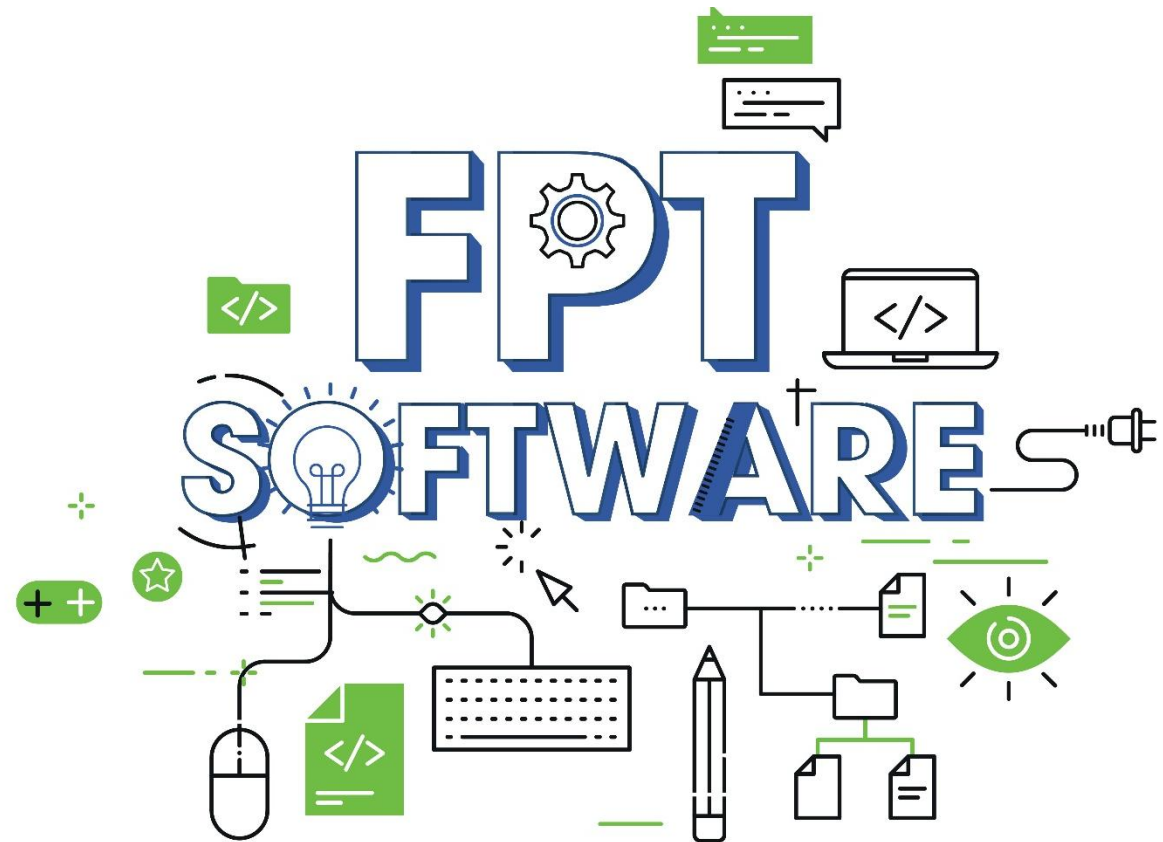


Basic Android

Saving data -SQLite Database



Agenda

1. **Save data using SQLite**

2. **Content Provider**



Lesson Objectives



01 Have basic understanding about Android's data storage

02 Have basic understanding about the ways to save data into SQLite

03 Have basic ability to create Android apps with applied SQLite database schema

- Most Android apps need to save data such as information about the app state, user settings or large amounts of information in files and databases.
- The principal data storage options in Android:
 - Saving key-value pairs of simple data types in a shared preferences file
 - Saving arbitrary files in Android's file system
 - Using databases managed by SQLite

- Saving Key-Value Sets
 - Learn to use a shared preferences file for storing small amounts of information in key-value pairs.
- Saving Files
 - Learn to save a basic file, such as to store long sequences of data that are generally read in order.
- Saving Data in SQL Databases
 - Learn to use a SQLite database to read and write structured data.

Save data using SQLite

Overview - Save data using SQLite

- Android default Database engine is Lite. SQLite is a lightweight transactional database engine that occupies a small amount of disk storage and memory, no network required, long-running operations and standalone.
- Saving data to a database is ideal for repeating or structured data, such as contact information. The APIs you'll need to use a database on Android are available in the [android.database.sqlite](#) package.
- Things to consider when dealing with SQLite:
 - ✓ Data type integrity is not maintained in SQLite, you can put a value of a certain data type in a column of another datatype (put string in an integer and vice versa).
 - ✓ Referential integrity is not maintained in SQLite, there is no FOREIGN KEY constraints or JOIN statements.
 - ✓ SQLite Full Unicode support is optional and not installed by default.

Define a schema and contract

- One of the main principles of SQL databases is the schema: a formal declaration of how the database is organized. The schema is reflected in the SQL statements that you use to create your database.
- A contract class is a container for constants that define names for URIs, tables, and columns.
- A good way to organize a contract class is to put definitions that are global to your whole database in the root level of the class. Then create an inner class for each table. Each inner class enumerates the corresponding table's columns.

Define a schema and contract

- First of all, we need to configure the schema and must implement the schema, such that it is one of the principles of SQL databases. It is very helpful for declaring the contract classes that explicitly specify the layout of the schema in a systematic and self-documenting manner.
- How to create the table and rows in an SQLite database:

```
object FeedReaderContract {  
    // Table contents are grouped together in an anonymous object.  
    object FeedEntry : BaseColumns {  
        const val TABLE_NAME = "entry"  
        const val COLUMN_NAME_TITLE = "title"  
        const val COLUMN_NAME_SUBTITLE = "subtitle"  
    }  
}
```

Create a database using an SQL helper

- Hear we have some statements that create and delete a table as following:

```
private const val SQL_CREATE_ENTRIES =  
    "CREATE TABLE ${FeedEntry.TABLE_NAME} (" +  
        "${BaseColumns._ID} INTEGER PRIMARY KEY," +  
        "${FeedEntry.COLUMN_NAME_TITLE} TEXT," +  
        "${FeedEntry.COLUMN_NAME_SUBTITLE} TEXT)"  
private const val SQL_DELETE_ENTRIES = "DROP TABLE IF EXISTS  
    ${FeedEntry.TABLE_NAME}"
```

- Just like files that save on the device's internal storage, Android stores database in app's private folder. Data is secure, because by default this area is not accessible to other apps or the user.

Create a database using an SQL helper

- SQLiteOpenHelper class provides the onCreate() and onUpgrade() methods for performing any database operation.
- Constructors of SQLiteOpenHelper Class:
 - ✓ SQLiteOpenHelper(Context context, String name, SQLiteDatabase.CursorFactory factory, int version): This constructor creates an object for creating, opening, and managing the database.
 - ✓ SQLiteOpenHelper(Context context, String name, SQLiteDatabase.CursorFactory factory, int version, DatabaseErrorHandler errorHandler): This constructor creates an object for creating, opening, and managing the database. It specifies the error handler.
- Methods of SQLiteOpenHelper class:
 - ✓ SQLiteOpenHelper class has many methods. Some of them are as follows:
 - **onCreate**: This method is called only when database created for the first time.
 - **onUpgrade**: This method is called when the database needs to be upgraded.
 - **close**: This method closes the database object.

Create a database using an SQL helper

- To use SQLiteOpenHelper, create a subclass that overrides the onCreate() and onUpgrade() callback method:
 - ✓ onCreate(SQLiteDatabase db): invoked when the database is created, this is where we can create tables and columns to them, create views or triggers.
 - ✓ onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion): invoked when we make a modification to the database such as altering, dropping , creating new tables.

```
class FeedReaderDbHelper(context: Context) : SQLiteOpenHelper(context, DATABASE_NAME, null, DATABASE_VERSION) {  
    override fun onCreate(db: SQLiteDatabase) {  
        db.execSQL(SQL_CREATE_ENTRIES)  
    }  
    override fun onUpgrade(db: SQLiteDatabase, oldVersion: Int, newVersion: Int) {  
        // This database is only a cache for online data, so its upgrade policy is  
        // to simply to discard the data and start over  
        db.execSQL(SQL_DELETE_ENTRIES)  
        onCreate(db)  
    }  
    override fun onDowngrade(db: SQLiteDatabase, oldVersion: Int, newVersion: Int) {  
        onUpgrade(db, oldVersion, newVersion)  
    }  
    companion object {  
        // If you change the database schema, you must increment the database version.  
        const val DATABASE_VERSION = 1  
        const val DATABASE_NAME = "FeedReader.db"  
    }  
}
```

Put information into a database

- Gets the data repository in write mode:

```
val db = dbHelper.writableDatabase
```

- ✓ Because they can be long-running, be sure that you call [getWritableDatabase\(\)](#) or [getReadableDatabase\(\)](#) in a background thread.

- Create a new map of values, where column names are the keys:

```
val values = ContentValues().apply {  
    put(FeedEntry.COLUMN_NAME_TITLE, title)  
    put(FeedEntry.COLUMN_NAME_SUBTITLE, subtitle)  
}
```

- ✓ Insert the new row, returning the primary key value of the new row.

```
val newRowId = db?.insert(FeedEntry.TABLE_NAME, null, values)
```

- ✓ The first argument for [insert\(\)](#) is table name.
- ✓ The second argument tells the framework what to do in the event that the [ContentValues](#) is empty (i.e., you did not [put](#) any values). If you specify the name of a column, the framework inserts a row and sets the value of that column to null.
- ✓ The [insert\(\)](#) methods returns the ID for the newly created row, or it will return -1 if there was an error inserting the data.

Read information from a database

- Gets the data repository in read mode:

```
val db = dbHelper.readableDatabase
```

- ✓ Because they can be long-running, be sure that you

call [getWritableDatabase\(\)](#) or [getReadableDatabase\(\)](#) in a background thread.

- Define a projection that specifies which columns from the database will actually use after this query:

```
val projection = arrayOf(BaseColumns._ID, FeedEntry.COLUMN_NAME_TITLE,  
    FeedEntry.COLUMN_NAME_SUBTITLE)
```

- ✓ Filter results **WHERE** "title" = 'My Title':

```
val selection = "${FeedEntry.COLUMN_NAME_TITLE} = ?"  
val selectionArgs = arrayOf("My Title")
```

- How you want the results sorted in the resulting Cursor:

```
val sortOrder = "${FeedEntry.COLUMN_NAME_SUBTITLE} DESC"  
val cursor = db.query(  
    FeedEntry.TABLE_NAME,    // The table to query  
    projection,              // The array of columns to return (pass null to get all)  
    selection,               // The columns for the WHERE clause  
    selectionArgs,          // The values for the WHERE clause  
    null,                   // don't group the rows  
    null,                   // don't filter by row groups  
    sortOrder                // The sort order
```

Update information from a database

- When need to modify a subset of your database values, use the `update()` method. Updating the table combines the `ContentValues` syntax of `insert()` with the WHERE syntax of `delete()`.
- Gets the data repository in write mode:

```
val db = dbHelper.writableDatabase
```

```
// New value for one column
val title = "MyNewTitle"
val values = ContentValues().apply {
    put(FeedEntry.COLUMN_NAME_TITLE, title)
}

// Which row to update, based on the title
val selection = "${FeedEntry.COLUMN_NAME_TITLE} LIKE ?"
val selectionArgs = arrayOf("MyOldTitle")
val count = db.update(
    FeedEntry.TABLE_NAME,
    values,
    selection,
    selectionArgs)
```

Delete information from a database

- To delete rows from a table, you need to provide selection criteria that identify the rows to the delete() method. The mechanism works the same as the selection arguments to the query() method.

```
// Define 'where' part of query.  
val selection = "${FeedEntry.COLUMN_NAME_TITLE} LIKE ?"  
// Specify arguments in placeholder order.  
val selectionArgs = arrayOf("MyTitle")  
// Issue SQL statement.  
val deletedRows = db.delete(FeedEntry.TABLE_NAME, selection, selectionArgs)
```

- The return value for the delete() method indicates the number of rows that were deleted from the database.

Persisting database connection

- Since `getWritableDatabase()` and `getReadableDatabase()` are expensive to call when the database is closed, you should leave your database connection open for as long as you possibly need to access it. Typically, it is optimal to close the database in the `onDestroy()` of the calling Activity.

```
override fun onDestroy() {  
    dbHelper.close()  
    super.onDestroy()  
}
```

Debug your database

- The Android SDK includes a sqlite3 shell tool that allows to browse table contents, run SQL commands, and perform other useful functions on SQLite databases.
- Using Shell:
 - ✓ Enable Developer option in Android device
 - ✓ Enable USB debugging
 - ✓ Connect your device via USB
 - ✓ When prompted 'Allow USB debugging. Click OK.
 - ✓ Unix Shell can be used to run a variety of commands on a device. Android Debug Bridge(adb) provides access to it. Open ADB's shell from the command prompt by running ADB shell. This shell can be used to copy the database out of the applications directory and paste it to any location on the device.

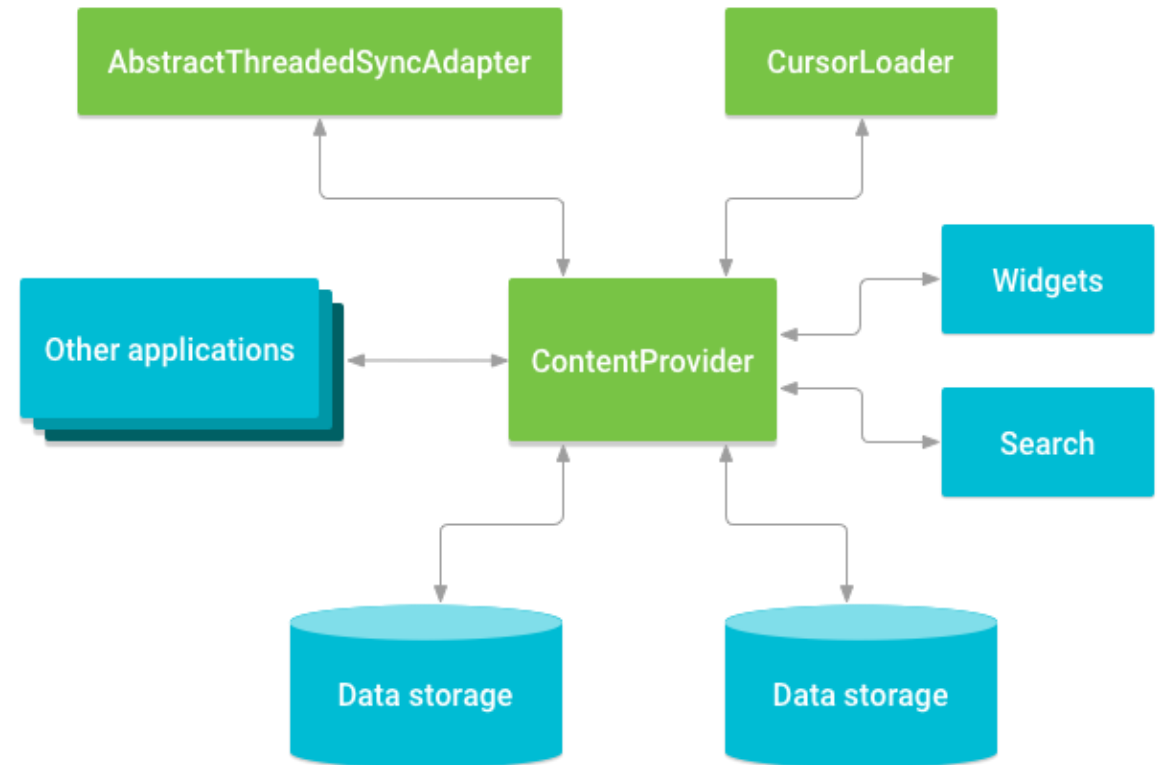
```
hell@device:/ $ mkdir /sdcard/data/  
shell@device:/$ run-as <com.your.package>  
shell@device:/data/data/com.your.package $ cp databases/<database>.db  
/sdcard/data/  
shell@device:/data/data/com.your.package $ exit  
shell@device:/ $ exit
```

- Now the database file is accessible from “/sdcard/data/”. Use ADB pull to pull the file to the localhost:
 - ✓ adb pull /sdcard/data/<database>.db

Content Provider

Overview

- A content provider presents data to external applications as one or more tables that are similar to the tables found in a relational database.
 - ✓ A row represents an instance of some type of data the provider collects,
 - ✓ Each column in the row represents an individual piece of data collected for an instance.



Retrieving Data from the Provider

- To retrieve data from a provider, follow these basic steps:
 - ✓ Request read access permission for the provider.
 - ✓ Define the code that sends a query to the provider.
- *android.permission.READ_USER_DICTIONARY* permission to retrieve data from it.
The provider has a separate
- *android.permission.WRITE_USER_DICTIONARY* permission for inserting, updating, or deleting data.
- The following sample `<uses-permission>` element requests read access to the User Dictionary Provider:

```
<uses-permission android:name="android.permission.READ_USER_DICTIONARY">
```

Retrieving Data from the Provider

- Constructing the query:

- ✓ The next step in retrieving data from a provider is to construct a query.

```
// A "projection" defines the columns that are returned for each row
private val mProjection: Array<String> = arrayOf(
    UserDictionary.Words._ID,      // Contract class constant for the _ID column name
    UserDictionary.Words.WORD,     // Contract class constant for the word column name
    UserDictionary.Words.LOCALE    // Contract class constant for the locale column name
)

// Defines a string to contain the selection clause
private var selectionClause: String? = null

// Declares an array to contain selection arguments
private lateinit var selectionArgs: Array<String>
```

Retrieving Data from the Provider

- Protect against malicious input

```
// Constructs a selection clause by concatenating the user's input to the column name  
var selectionClause = "var = $mUserInput"
```

- Let the user potentially concatenate malicious SQL onto your SQL statement.
- To avoid this problem, use a selection clause that uses ? as a replaceable parameter and a separate array of selection arguments

```
// Constructs a selection clause with a replaceable parameter  
var selectionClause = "var = ?"
```

Create a Content Provider

- Follow these steps to build provider:
 1. Design the raw storage for data. A content provider offers data in two ways:
 - ✓ File data: Data that normally goes into files, such as photos, audio, or videos. Store the files in your application's private space.
 - ✓ "Structured" data: Data that normally goes into a database, array, or similar structure. Store the data in a form that's compatible with tables of rows and columns.
 2. Define a concrete implementation of the ContentProvider class and its required methods.
 3. Define the provider's authority string, content URIs, and column names. If you want the provider's application to handle intents, also define intent actions, extras data, and flags. Also define the permissions that you require for applications that want to access data.
 4. Add other optional pieces ...

Implement the Content Provider class

- The [ContentProvider](#) instance manages access to a structured set of data by handling requests from other applications. All forms of access eventually call [ContentResolver](#), which then calls a concrete method of ContentProvider to get access.
- Required methods:
 - ✓ [query\(\)](#)
Retrieve data from your provider. Use the arguments to select the table to query, the rows and columns to return, and the sort order of the result. Return the data as a [Cursor](#) object.
 - ✓ [insert\(\)](#)
Insert a new row into your provider. Use the arguments to select the destination table and to get the column values to use. Return a content URI for the newly inserted row.
 - ✓ [update\(\)](#)
Update existing rows in your provider. Use the arguments to select the table and rows to update and to get the updated column values. Return the number of rows updated.
 - ✓ [delete\(\)](#)
Delete rows from your provider. Use the arguments to select the table and the rows to delete. Return the number of rows deleted.
 - ✓ [getType\(\)](#)
Return the MIME type corresponding to a content URI. This method is described in more detail in the [Implement content provider MIME types](#) section.
 - ✓ [onCreate\(\)](#)
Initialize your provider. The Android system calls this method immediately after it creates your provider. Your provider isn't created until a [ContentResolver](#) object tries to access it.
- These methods have the same signature as the identically named ContentResolver methods.

Design content URIs

- A content URI is a URI that identifies data in a provider. Content URIs include the symbolic name of the entire provider (its authority) and a name that points to a table or file (a path). The optional ID part points to an individual row in a table. Every data access method of [ContentProvider](#) has a content URI as an argument. This lets you determine the table, row, or file to access.
- **Design an authority:** A provider usually has a single authority, which serves as its Android-internal name.

Example: if your Android package name is *com.example.<appname>*, give your provider the authority *com.example.<appname>.provider*.

- **Design a path structure:** Developers usually create content URIs from the authority by appending paths that point to individual tables.

Example:

com.example.<appname>.provider/table1 and *com.example.<appname>.provider/table2*

Content Provider Permissions

- A provider's application can specify permissions that other applications must have in order to access the provider's data.
 - ✓ End users see the requested permissions when they install the application.
- If a provider's application doesn't specify any permissions, then other applications have no access to the provider's data.
 - ✓ However, components in the provider's application always have full **read** and **write access**.
- To get the permissions, an application requests them with a **<uses-permission>** element in its manifest file.

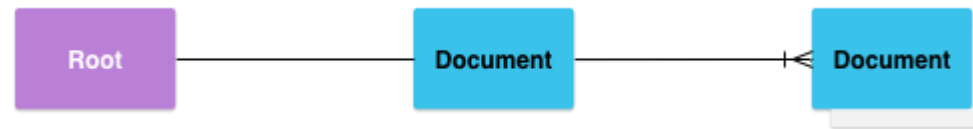
Content Provider Permissions

- The following list describes the scope of provider permissions, starting with the permissions that apply to the entire provider and then becoming more fine-grained. More fine-grained permissions take precedence over ones with larger scope.
 - ✓ Single read-write provider-level permission
 - ✓ Separate read and write provider-level permissions
 - ✓ Path-level permission
 - ✓ Temporary permission

Open files using the Storage Access Framework



- Android 4.4 (API level 19) introduces the Storage Access Framework (**SAF**). The SAF lets users browse and open documents, images, and other files across all of their preferred document storage providers
- The SAF centers around a content provider that is a subclass of the [DocumentsProvider](#) class. Within a document provider, data is structured as a traditional file hierarchy:



- The **SAF** includes the following:
 - ✓ **Document provider**: a content provider that lets a storage service, such as Google Drive, reveal the files it manages. A document provider is implemented as a subclass of the [DocumentsProvider](#) class.
 - ✓ **Client app**: invokes the [ACTION_CREATE_DOCUMENT](#), [ACTION_OPEN_DOCUMENT](#), and [ACTION_OPEN_DOCUMENT_TREE](#) intent actions and receives the files returned by document providers
 - ✓ **Picker**: a system UI that lets users access documents from all document providers

Open files using the Storage Access Framework



- On Android 4.3 and lower, to retrieve a file from another app, invoke an intent such as [ACTION_PICK](#) or [ACTION_GET_CONTENT](#). The user then selects a single app from which to pick a file. The selected app must provide a user interface for the user to browse and pick from the available files.
- On Android 4.4 (API level 19) and higher, using the [ACTION_OPEN_DOCUMENT](#) intent action allows users to select a specific document or file to open.
- On Android 5.0 (API level 21) and higher, use the [ACTION_OPEN_DOCUMENT_TREE](#) intent allows users to select a specific directory, granting your app access to all of the files and sub-directories within that directory.

Open files using the Storage Access Framework



- Grant access to a directory's contents
- Optionally, specify the URI of the directory that the file picker should display when it first loads by using the EXTRA_INITIAL_URI intent extra.
- The following code snippet shows how to create and invoke the intent for opening a directory:

```
fun openDirectory(pickerInitialUri: Uri) {  
    // Choose a directory using the system's file picker.  
    val intent = Intent(Intent.ACTION_OPEN_DOCUMENT_TREE).apply {  
        // Optionally, specify a URI for the directory that should be opened in  
        // the system file picker when it loads.  
        putExtra/DocumentsContract.EXTRA_INITIAL_URI, pickerInitialUri)  
    }  
    startActivityForResult(intent, your-request-code)  
}
```

Open files using the Storage Access Framework



- After the user has selected a file or directory using the system's file picker, retrieve the selected item's URI using the following code in [onActivityResult\(\)](#):

```
override fun onActivityResult(  
    requestCode: Int, resultCode: Int, resultData: Intent?) {  
    if (requestCode == your-request-code  
        && resultCode == Activity.RESULT_OK) {  
        // The result data contains a URI for the document or directory that  
        // the user selected.  
        resultData?.data?.also { uri ->  
            // Perform operations on the document using its URI.  
        }  
    }  
}
```

- By getting a reference to the selected item's URI, your app can perform several operations on the item. For example, we can access the item's metadata, edit the item in place, and delete the item.

Create a custom document provider

- Purpose of create a custom document provider:
 - ✓ Offer complex data or files to other applications.
 - ✓ Provides storage services for files (such as a cloud save service)
- To implement a custom document provider, add the following to application's manifest:

```
<manifest... >
  ...
  <uses-sdk
    android:minSdkVersion="19"
    android:targetSdkVersion="19" />
    ....
    <provider
      android:name="com.example.android.storageprovider.MyCloudProvider"
      android:authorities="com.example.android.storageprovider.documents"
      android:grantUriPermissions="true"
      android:exported="true"
      android:permission="android.permission.MANAGE_DOCUMENTS">
        <intent-filter>
          <action android:name="android.content.action.DOCUMENTS_PROVIDER" />
        </intent-filter>
      </provider>
    </application>
  </manifest>
```

Create a custom document provider

- The SAF provides these contract classes:
 - ✓ DocumentsContract.Document
 - ✓ DocumentsContract.Root
- For example:

```
private val DEFAULT_ROOT_PROJECTION: Array<String> = arrayOf(  
    DocumentsContract.Root.COLUMN_ROOT_ID,  
    DocumentsContract.Root.COLUMN_MIME_TYPES,  
    DocumentsContract.Root.COLUMN_FLAGS,  
    DocumentsContract.Root.COLUMN_ICON,  
    DocumentsContract.Root.COLUMN_TITLE,  
    DocumentsContract.Root.COLUMN_SUMMARY,  
    DocumentsContract.Root.COLUMN_DOCUMENT_ID,  
    DocumentsContract.Root.COLUMN_AVAILABLE_BYTES  
)  
private val DEFAULT_DOCUMENT_PROJECTION: Array<String> = arrayOf(  
    DocumentsContract.Document.COLUMN_DOCUMENT_ID,  
    DocumentsContract.Document.COLUMN_MIME_TYPE,  
    DocumentsContract.Document.COLUMN_DISPLAY_NAME,  
    DocumentsContract.Document.COLUMN_LAST_MODIFIED,  
    DocumentsContract.Document.COLUMN_FLAGS,  
    DocumentsContract.Document.COLUMN_SIZE  
)
```

Create a custom document provider

- The next step in writing a custom document provider is to subclass the abstract class DocumentsProvider, implement following methods:
 - ✓ queryRoots()
 - ✓ queryChildDocuments()
 - ✓ queryDocument()
 - ✓ openDocument()

Get document information

- The implementation of queryDocument() must return a Cursor that points to the specified file, using columns defined in DocumentsContract.Document.
- The queryDocument() method returns the same information that was passed in queryChildDocuments(), but for a specific file:

```
override fun queryDocument(documentId: String?, projection: Array<out String>?):  
Cursor {  
    // Create a cursor with the requested projection, or the default projection.  
    return MatrixCursor(resolveDocumentProjection(projection)).apply {  
        includeFile(this, documentId, null)  
    }  
}
```

Open document information

- Implement `openDocument()` to return a `ParcelFileDescriptor` representing the specified file. Other apps can use the returned `ParcelFileDescriptor` to stream data.
- The client app requests access to it by calling `openFileDescriptor()`

```
override fun openDocument(  
    documentId: String,  
    mode: String,  
    signal: CancellationSignal  
): ParcelFileDescriptor {  
    Log.v(TAG, "openDocument, mode: $mode")  
    // It's OK to do network operations in this method to download the document,  
    // as long as you periodically check the CancellationSignal. If you have an  
    // extremely large file to transfer from the network, a better solution may  
    // be pipes or sockets (see ParcelFileDescriptor for helper methods).  
  
    val file: File = getFileForDocId(documentId)  
    val accessMode: Int = ParcelFileDescriptor.parseMode(mode)  
  
    val isWrite: Boolean = mode.contains("w")  
    return if (isWrite) {  
        val handler = Handler(context.mainLooper)  
        // Attach a close listener if the document is opened in write mode.  
        try {  
            ParcelFileDescriptor.open(file, accessMode, handler) {  
                // Update the file with the cloud server. The client is done  
                writing.  
                Log.i(TAG, "A file with id $documentId has been closed! Time to  
                update the server.")  
            }  
        } catch (e: IOException) {  
            throw FileNotFoundException(  
                "Failed to open document with id $documentId and mode $mode"  
            )  
        }  
    } else {  
        ParcelFileDescriptor.open(file, accessMode)  
    }  
}
```

THANK YOU!

